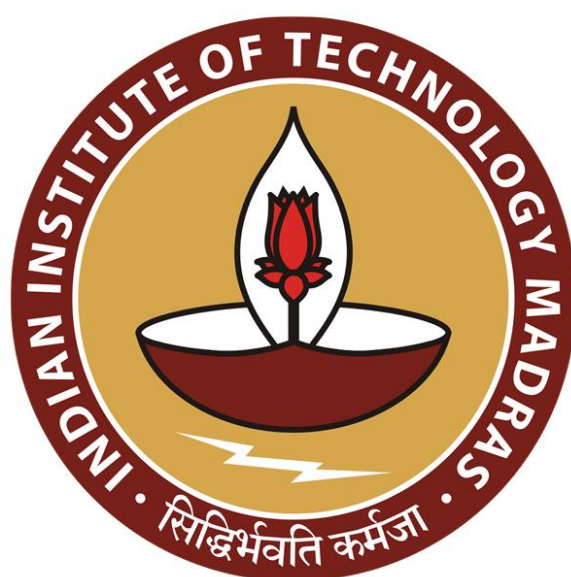




IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Week 10
Mutual Exclusion

So, we are looking at this Mutual Exclusion problem. The problem of ensuring that there are no race conditions when we try to concurrently update data.

(Refer Slide Time: 00:23)

Mutual exclusion

- Concurrent update of a shared variable can lead to data inconsistency
 - Race condition
- Control behaviour of threads to regulate concurrent updates
 - Critical sections — sections of code where shared variables are updated
 - Mutual exclusion — at most one thread at a time can be in a critical section

Madhavan Mukund

Mutual Exclusion

Programming Concepts using Java 2/7

So, remember that a race condition is one where we have data which is shared and which is being updated or viewed from two concurrent threads and because of concurrent updates, we might see inconsistent values of the data. So, for this we said we will identify these critical sections where this concurrent data is being manipulated or observed and ensure that there is mutual exclusion. So, how do you, how do you achieve this kind of mutual exclusion?

(Refer Slide Time: 00:50)

IIT Madras BSc Degree

Mutual exclusion for two processes

■ First attempt

```
Thread 1
...
while (turn != 1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 2;
...

Thread 2
...
while (turn != 2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 1;
...
```

Bank acct

Madhavan Mahand Mutual Exclusion Programming Concepts using Java 3/7

IIT Madras BSc Degree

Mutual exclusion for two processes

■ First attempt

```
Thread 1
...
while (turn != 1){ ✓
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 2;
...

Thread 2
...
while (turn != 2){ ✓
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 1;
...
```

■ Shared variable turn — no assumption about initial value, atomic update

Madhavan Mahand Mutual Exclusion Programming Concepts using Java 3/7

Mutual exclusion for two processes

First attempt

```
Thread 1
...
while (turn != 1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 2;
...
```

```
Thread 2
...
while (turn != 2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 1;
...
```

Out of action

- Shared variable **turn** — no assumption about initial value, atomic update
- Mutually exclusive access is guaranteed ...



Madhavan Mukund

Mutual Exclusion

Programming Concepts using Java

3/7

Mutual exclusion for two processes

First attempt

```
Thread 1
...
while (turn != 1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 2;
...
```

```
Thread 2
...
while (turn != 2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
turn = 1;
...
```

- Shared variable **turn** — no assumption about initial value, atomic update
- Mutually exclusive access is guaranteed ...
- ...but one thread is locked out permanently if other thread shuts down
- Starvation!**



Madhavan Mukund

Mutual Exclusion

Programming Concepts using Java

3/7

So, let us look at the simplest case I have two concurrent threads and there is some critical section. So, I am not at this point going to be worried about what specifically critical section is. So, this could be that bank account update. So, in general there is something in thread 1 and something in thread 2 which manipulates some kind of data and I want to ensure that both these manipulations do not interfere with each other.

And what we said is that if we can ensure that at most one of them is inside that at a time, as though thread 1 happened before thread 2 or thread 2 happened before thread 1. So, there is no interference and therefore this kind of a race condition is totally avoided. So, what we have here is, if you notice, the same value turn on the other side.

So, what is happening in this protocol as we call it, is that the first thread, thread 1 is, so, the turn is, so, each thread is trying to be nice to the other thread. So, it is trying to say it is your turn. So, if it is the turn of thread 1, then turn will be 1. So, as long as it is not 1, thread 1 will wait and when turn becomes 1, it will execute and then it will hand over to the other guy.

Symmetrically, thread 2 will wait for its turn and when it is done it will hand over the turn to the other guy. So, so, the nice way would be that thread 1 sees turn equal to 1, it executes and says turn equal to 2. So, now when turn, the thread 2 which is waiting, so, what we have written as a busy, wait, basically, says that this thread is executing this condition repeatedly without making any progress.

So, it is not that it is being told that now the value is something that is useful for you, but rather it is checking it. So, it is repeatedly checking. So, this is how we envisage also this browser, which is downloading. So, periodically when it is downloading, we said that that download thread will be checking whether terminate has been set to true or not. So, once in a while, it has to interrupt whatever is doing there to check it. Here we are just saying is doing nothing. So, this is it is busy, waiting. So, that is what a busy wait is.

So, it is busy, doing nothing, waiting for some good condition to happen. So, what we have here? Therefore is we have a shared variable turn, but it is important that we are not making any assumptions, because we are trying to implement after all, some kind of atomic access, something which avoids race conditions.

So, we cannot say, let us assume that we have a turn which is nice and then based on that nice turn we will implement it. Because, how do you get turn to begin? So, turn has to be a regular variable. It is a shared variable with no assumptions. So, it could be that in the while you are trying to set turn to 1 while this is happening.

Something happens and this one gets it, so, you try to set turn equal to 1 but actually the value turns out to be 2 because the other thread has come and overwritten. So, we do not make any assumptions about turn itself being a well-behaved variable, nor do we care how turn is set, except we know that it is either 1 or 2, is nothing else.

Because, thread 1 sets it to 1 thread 2 sets it to 2. So, you can assume if you want that thread is a Boolean is either false or true, and false means 1 and true means 2. So, it is a two-valued

variable but we do not know whether it is 1 or 2 to begin with. But, still as we said, because it can only be one of those two values, exactly one of these two whiles will succeed.

So, if both of them are trying to wait to get in; it is either 1 or it is 2. So, one of them will say it is not equal and will continue the busy, wait, the other one will say it is equal and the while will exit and it will proceed. And because of this kind of switch over, if the while succeeded because it was 2, the value will be set to 1 and so the other guys while will now succeed and they will be let through.

So, using this kind of reasoning, you have to formalize it little bit to get a formal proof but you can easily argue that under these assumptions, the only assumption we are making is that this turn has values 1 and 2 to begin with. So, one of them, so, if it has a value like 0, then both of them will be waiting forever nobody is going to set it to 1 or 2 for them. So, that is not a good assumption. So, it is either 1 or 2 to start with. We do not care which. So, one of them will be able to get in and then once they get in they can switch each other back and forth.

So, mutually exclusive access is guaranteed. But there is a problem. So, supposing this guy is out of action for whatever reason. So, thread 2 is not executing or is not interested in doing whatever it wants. So, it is got kind of it is currently not an active thread. So, this guy gets in, sets turn equal to 2 and then he comes back and now is waiting again. But having said turn equal to 2, the only way that this thread can get back in is for the other thread to cooperate and set turn back to 1, but the other thread is out of action.

So, if this thread is alone trying to access then this solution fails and this is something which is called starvation, that is I could be stuck, waiting for the resource. Because, the other process, which is supposed to cooperate to give me the resource is not cooperating. And in this case is not cooperating by doing something very simple, it is just not active. It is not active.

It is not trying to not cooperate. It is just that it does not have any interest. So, we are really saying that this solution will work only if these both these threads are continuously interested in this resource, in which case they will keep alternating. 1 will give it to 2, 2 will give it to 1 and so on. But if this continuous alternation ever fails, then the one that is waiting is going to

get stuck until the other guy comes back. So, this is not really a valid solution. So, we could use a different solution which is based again on a shared variable.

(Refer Slide Time: 06:42)

IFT Madras
BSc Degree

Mutual exclusion for two processes ...

■ Second attempt

```
Thread 1
...
request_1 = true;
while (request_2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_1 = false;
...

Thread 2
...
request_2 = true;
while (request_1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_2 = false;
...
```

Madhavan Mukund

Mutual Exclusion

Programming Concepts using Java 4/7

IFT Madras
BSc Degree

Mutual exclusion for two processes ...

■ Second attempt

```
Thread 1
...
request_1 = true;
while (request_2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_1 = false;
...

Thread 2
...
request_2 = true;
while (request_1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_2 = false;
...
```

■ Mutually exclusive access is guaranteed ...

Madhavan Mukund

Mutual Exclusion

Programming Concepts using Java 4/7

■ Second attempt

Thread 1	Thread 2
...	...
request_1 = true;	request_2 = true;
while (request_2){	while (request_1){
// "Busy" wait	// "Busy" wait
}	}
// Enter critical section	// Enter critical section
...	...
// Leave critical section	// Leave critical section
request_1 = false;	request_2 = false;
...	...

■ Mutually exclusive access is guaranteed ...

■ ... but if both threads try simultaneously, they block each other

■ Deadlock!



But these are now two shared variables. So, there we had a single variable turn, which was set to either 1 or 2. Here, we will have two Boolean variables, signaling that each of them has made a request. So, if the first thread wants to get into the, again we have this critical section which we are trying to protect. So, that is common.

And again, we are going to do some waiting, but what we are going to have is these two variables request 1 and request 2. So, request 1 says that thread 1 is interested in getting into critical section; request 2 says that thread 2 is interested in getting the critical section. And now just like there we had this politeness I give you my turn, you give me my turn back, I give you a turn, you give your turn back.

So, here if I will, I am interested, but the other guy is also interested, then I wait. So, I wait here so long as I am interested, but 2 is also interested. Now, when 2 finishes, 2 is going to release its interest. So, it is going to say I am no longer interested and then this will clear and you will get through. So, 1 waits for 2 to release its interest and 2 waits for 1 to release its interest. And this is done through two Boolean variables.

So, these are again shared but they are shared in a different way. Earlier both processes were writing turn and reading turn. They were checking the value of turn and they were updating turn to 1 and 2. Here, one is reading one is writing. So, request 1 is written only by 1, but read by 2 and request 2 is written only by 2 and read by 1, but still these are shared like that transfer and audit.

Only transfer is updating accounts, but audit still had a role to play in reading the accounts. So, same way. So, again you can reason about this and argue that both of them will never simultaneously get in, because request 1 and request 2 are never going to be. So, here in the earlier case, it was a little easier to argue because turn was either 1 or 2. So, if it is either 1 or 2, both those conditions cannot be true. It cannot be that turn is not equal to 1 and turn is not equal to 2 at the same time.

Here, it is a little more tricky, because we have request 1 and request 2. We have two different variables and they are independent of each other. So, there is no reason why both of them should not become false at the same time. So, you have to do a little bit more of argument to make sure this correct. But, you can argue that mutually exclusive access is guaranteed.

But, now what happens, if these processes both come here and reach this at the same point. So, they both decide that they want access to the critical section. So, they both turn on their requests. So, at this point now when thread 1 tries to look at request 2, it will find that request 2 has been set to true and so thread 1 will have to wait. And symmetrically, thread 2 will look at thread 1 and say, oh, thread 1 is way is interested. So, I will also wait.

So, earlier we had a situation that we were waiting for the other person to cooperate and release us. Here, we have a situation that both of them think the other one is interested. So, they are both kind of standing at the door and say, you first, you first, and neither of them is making any progress. So, this is a different problem from the earlier one.

The problem the earlier one was that only one was interested, but on its own it could not proceed and therefore we had the starvation. Here now, both are simultaneously interested and they cannot decide on who is going to go next and. So, they are stuck. So, this is called a deadlock.

So, these are the two typical problems that you have when you are dealing with this kind of critical section access, that is you have a situation, where it works fine in most cases, but in some cases one of them gets locked out. So, that is starvation. Or in one situation, two of them are contending and there is no way to resolve this. And that is deadlock. So, how do we get around this?

(Refer Slide Time: 10:24)

Peterson's algorithm

```

Thread 1
...
request_1 = true;
turn = 2;
while (request_2 &&
    turn != 1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_1 = false;
...

Thread 2
...
request_2 = true;
turn = 1;
while (request_1 &&
    turn != 2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_2 = false;
...

```

- Combines the previous two approaches



Peterson's algorithm

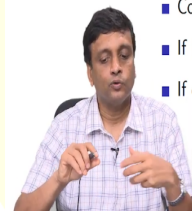
```

Thread 1
...
request_1 = true;
turn = 2;
while (request_2 &&
    turn != 1){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_1 = false;
...

Thread 2
...
request_2 = true;
turn = 1;
while (request_1 &&
    turn != 2){
    // "Busy" wait
}
// Enter critical section
...
// Leave critical section
request_2 = false;
...

```

- Combines the previous two approaches
- If both try simultaneously, **turn** decides who goes through
- If only one is alive, **request** for that process is stuck at false and **turn** is irrelevant



Well, it turns out that there is a very clever solution due to Peterson, which combines these two things and together they work. So, individually the turn based solution has starvation and the request 1 request 2 solution has got deadlock. But, you combine them and magically it works. So, here we have both. So, we have the turn variable being set to 1 or 2.

So, remember that in the earlier case I set the turn at the end after I finished my work process 1 or thread 1 said turn equal to 2 and after thread 2 finished its work it set it to 1. So, here up front I do that. So, up front I say it is your turn. But I also say I am interested. So, there are these two things. So, this one says request 1 is true but is your turn. This one says request 2 is true but it is your turn.

And now you check both. You check whether the other person is interested or it is my turn. So, if it is not my turn I cannot proceed and if request 2 is interested and not proceed but either of them fails I can, I get blocked only both are, so, this is the logical AND. So, this says that I will not go through this provided the other thread is interested and the turn is with the other thread.

So, either if the other thread is not interested or if the turn is with me, I will go through. And then at the end of this, of course, we reset this request saying that I am no longer interested. So, the turn was set at the beginning and this request is reset at the end as before. So, why does this work? Well, again it requires an argument. But intuitively, what is happening is that if both of them try which is the deadlock situation. Then because turn has to be 1 or 2, one of these conditions will end up becoming false.

So, whoever has the turn will be able to go through, even though both of them are requesting. So, the deadlock is no longer an issue. On the other hand, if I have this condition, where I am waiting for my turn but the turn is not coming to me, if this guy is not interested, he must have gone past this and you must be there. So, this request must be false. The situation of starvation came in the turn-based solution, when the other thread refused to cooperate because it was just not interested, so, it was not obliging by turning the turn back to me.

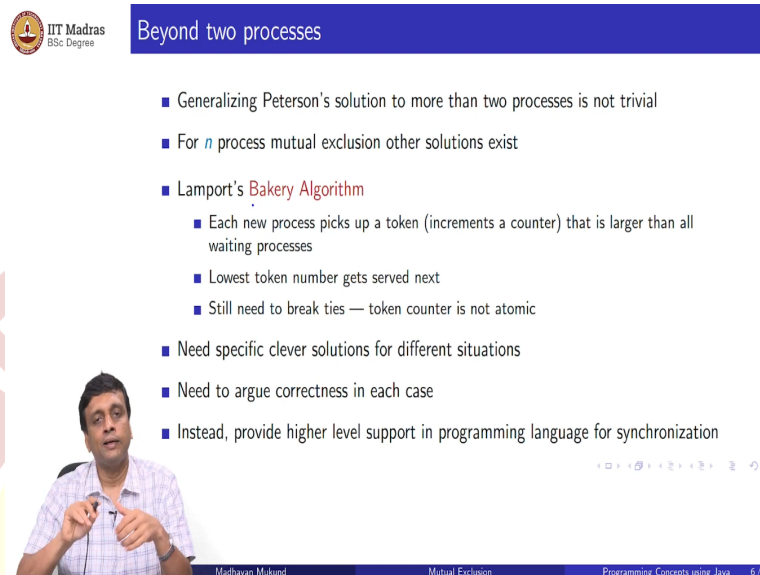
So, here it is fine. The turn does not come back to me because I have said given the person the turn but the request is certainly false. So, therefore while request 2 will be false and so, again the while condition will fail and I will be able to make progress. So, the control of the, so, the request which created a deadlock helps us to overcome starvation and the turn which created starvation helps us to overcome deadlock and together these things actually work for mutual exclusion of two processes.

So, Peterson's algorithm is a very clever combination of these two ideas, a very elegant combination, which works to solve this mutual exclusion problem. But, remember the important thing is there are no assumptions. In the sense, that turn request 1 request 2, they are well typed, in the sense, that request 1 request 2, a Boolean and turn is either 1 or 2.

But, other than that there is no assumption about their initial values and there is no assumption that they are updated atomically. They are just normal values which are shared

and updated in the usual race condition way, but by using these values and checking them, I am able to control access to this critical section.

(Refer Slide Time: 13:56)



IIT Madras
BSc Degree

Beyond two processes

- Generalizing Peterson's solution to more than two processes is not trivial
- For n process mutual exclusion other solutions exist
- Lamport's **Bakery Algorithm**
 - Each new process picks up a token (increments a counter) that is larger than all waiting processes
 - Lowest token number gets served next
 - Still need to break ties — token counter is not atomic
- Need specific clever solutions for different situations
- Need to argue correctness in each case
- Instead, provide higher level support in programming language for synchronization

Madhavan Mukund Mutual Exclusion Programming Concepts using Java 6/7

So, these are solutions that we have seen for two processes. Now, you can see that if you go for three processes it is not so easy to imagine what should be the case. Should I have a turn which has 1, 2, 3? Should I have one turn variable for every pair? Should I have three requests? And so on.

So, generalizing the solution to more than two processes is not very easy. In general, if I want to talk about more than two processes, there are other solutions other than Peterson's solution. The most well-known of these is what is called the Bakery Algorithm due to Lamport. So, it is called a bakery but in our context, it is more common when we go into a bank.

So, in an Indian context in most banks, if you go in, you pick up a token. And there will be a display and that display will tell you which token number has to go to which teller or which window. So, each person gets a new token. Now, of course, the token dispenser in a bank is an object, it is a physical thing and there is only one of them.

So, the customers who come to this token will necessarily get tokens in sequence. So, each person will get a token which has one number, a number one larger than the previous one. So, therefore you are guaranteed that if you process these tokens as they come then each customer will eventually get a turn and no two customers will have the same token number.

So, there would not be any clash. But, what if we do not have this token dispenser. So, that is what the Bakery algorithm says. The Bakery algorithm says that you walk into the bank. There is no token dispenser. You ask everybody, who is waiting, what their token number is. And you assign yourself a token which is one more than that.

So, you create a new token number based on the current token numbers of those who are waiting. And then you follow the same process, the person with the smallest token number, who is currently waiting, will be served next. But, notice that while I am processing, supposing there are seven people waiting and I am asking seven people, what their token number is? And other customer may walk in and do the same thing.

So, they may ask all these seven people the same thing. So, both of us will end up incrementing our token number to the same value which is one more than the maximum currently. So, there also has to be some mechanism to break ties in such a case showing that I came first therefore I am ahead of you and so on. So, Lamport's bakery algorithm has this tie-breaking rule and you can actually make it work for n processes.

But now, you can see that you know this is not really a scalable solution from our perspective. Every time we want to concur program a bank account which has concurrent access or something if we have to start imagining Bakery algorithm or we have to imagine this then we need a clever solution for each situation and we need to argue that what we have done is correct. It is not obvious that is correct.

So, it may be we may be lucky and we might be applying it to situation which you already seen before, but in general this requires a lot of tedious kind of manual design of protocols. So, while these are very nice protocols to understand and these are typically used in specific situations like an operating systems and all that it is not something that we want to burden ourselves with when we are doing actual programming.

So, what we would really like is that we are working in some programming language; this programming language should give us a slightly more high-level way of saying do this and do not allow other people to interfere. So, we would like some programming language support directly for synchronization.

(Refer Slide Time: 17:14)

- We can construct protocols that guarantee mutual exclusion to critical sections
 - Watch out for **starvation** and **deadlock**
- These protocols cleverly use regular variables
 - No assumptions about initial values, atomicity of updates
- Difficult to generalize such protocols to arbitrary situations
- Look to programming language for features that control synchronization



So, this is what we are going to come to next. So, to summarize what we have seen is that when we are designing protocols for mutual exclusion, we can do it with conventional shared variables where we do not have any assumptions about atomic update or initial values and so on.

But, if you are not careful we could run into two problems starvation where one of the processes is locked out because of non-cooperation or something and the other one is deadlock where two or more processes are contending and there is no way to decide who should go first .

So, using regular variables we came up with Peterson's algorithm which actually solves this for two processes without any assumptions about the variables. They are just shared variables of a normal type and is guaranteeing no starvation, no deadlock and mutual access, mutually exclusive access, to the critical section.

And Lamport solution for instance tells us with this bakery counter based algorithm like the bank token example, how to do it for n processes. But, what we said is that these are tricky and difficult to use and implement. So, what we would really like is programming language support for synchronization.